SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	**CSA1223**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL3- DEBUGGING & OPTIMISATION TASK**
Weightage:	**20%**	**Total Marks:**	**100**
CO2 Statement:	*Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)*		
Student Name:	**R DINESH KUMAR**	**Reg.No.:**	**192312258**
Department:	**ECE**	**Date:**	**12/08/26**

ANNEXURE A

1. **A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.**

Debugging Signed Integer Addition Using 2's Complement

In an 8-bit signed number system, the range of values is from **–128 to +127**. The given values are +120 and +50.

First, represent the numbers in binary:

+120 = `01111000`
+50 = `00110010`

Adding them:

```
  01111000   (+120)
+ 00110010   (+50)
-----------
  10101010
```

The result obtained is `10101010`. Since the most significant bit is 1, it represents a negative number in 2's complement. Therefore, the result is incorrect because the actual mathematical answer is:

120 + 50 = 170

But +170 cannot be represented using an 8-bit signed number.

The error is therefore **signed integer overflow**. Overflow occurs when two positive numbers are added and the result becomes negative, or when two negative numbers are added and the result becomes positive.

To detect overflow, the sign bits of the operands and result can be checked. If two numbers having the same sign produce a result with the opposite sign, overflow has occurred.

For this example:

```
Sign of 120 = 0
Sign of 50  = 0
Sign of result = 1
```

Hence, overflow is present.

Corrected Solution

The processor should check the overflow flag after signed addition. If overflow occurs, the system should report the condition instead of using the incorrect result.

For applications requiring larger values, a **16-bit or 32-bit integer** can be used. For example, using 16-bit representation, +170 can be represented correctly.

Testing

Test cases should include:

- 50 + 30 → 80, no overflow
- 120 + 50 → overflow
- –100 + –50 → overflow
- 100 + –50 → 50, no overflow

Thus, checking the overflow condition prevents incorrect signed arithmetic results.

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

A ripple carry adder performs addition by passing the carry from one bit to the next. For example, in a 4-bit adder, the carry generated at the first bit must travel through the remaining bits before the final result is obtained.

The main problem is **carry propagation delay**.

For example:

```
A0 + B0 → C1
C1 + A1 + B1 → C2
C2 + A2 + B2 → C3
C3 + A3 + B3 → C4
```

Each stage waits for the carry from the previous stage. Therefore, when the number of bits increases, the delay also increases. This becomes a problem in real-time systems where fast arithmetic operations are required.

Debugging the Bottleneck

The delay is mainly caused by the sequential propagation of carry through each full-adder stage. Even if the higher-order bits are ready, they cannot produce their final result until the carry arrives.

Optimized Solution

A **Carry Look-Ahead Adder (CLA)** reduces this delay by calculating the carry signals in advance.

For each bit:

Generate:

$$G_i = A_i \times B_i$$

Propagate:

$$P_i = A_i \oplus B_i$$

The carry can then be calculated directly:

$$C_1 = G_0 + P_0C_0$$
$$C_2 = G_1 + P_1G_0 + P_1P_0C_0$$

and similarly for the following bits.

Therefore, the carry does not need to wait for every previous stage.

Advantage

The main advantage of CLA is its **lower carry propagation delay** compared with a ripple carry adder. It is therefore more suitable for high-speed processors and real-time applications.

Testing

The CLA should be tested with different combinations of:

- No carry
- Carry from lower bits
- Carry through all bits
- Maximum possible values

Thus, replacing the ripple carry adder with a carry look-ahead adder improves arithmetic execution speed.

3. **A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.**

Booth's algorithm is used for multiplying signed binary numbers in 2's complement form. It reduces the number of addition and subtraction operations by examining pairs of bits.

The important bit pair is **Q0 and Q-1**.

The operations are:

Q0	Q-1	Operation
0	0	No operation
0	1	Add multiplicand
1	0	Subtract multiplicand
1	1	No operation

After each operation, an **arithmetic right shift** is performed on the combined registers.

Incorrect results for negative operands are usually caused by incorrect bit-pair checking, using a logical shift instead of an arithmetic shift, or failing to preserve the sign bit.

Debugging Steps

First, the multiplicand and multiplier must be represented using the same number of bits in 2's complement form.

The initial values of:

- Accumulator (A)
- Multiplier (Q)
- Multiplicand (M)
- Extra bit (Q-1)

must be initialized correctly.

During every iteration, `Q0` and `Q-1` must be checked before performing the required operation.

For a `10` pair, subtraction of the multiplicand is performed. For a `01` pair, the multiplicand is added.

After this, an **arithmetic right shift** is required. In an arithmetic shift, the leftmost sign bit is copied into the new position. This is important for negative numbers.

Possible Error

If a logical right shift is used, zero is inserted at the left side. This destroys the sign information and produces an incorrect result for negative operands.

Correction

The algorithm should:

1. Use proper 2's complement representation.
2. Check `Q0Q-1` correctly.
3. Perform addition or subtraction as required.
4. Use arithmetic right shift.
5. Preserve the sign bit.
6. Repeat the process for the required number of bits.

Testing

The algorithm should be tested with:

- Positive × positive
- Positive × negative
- Negative × positive
- Negative × negative

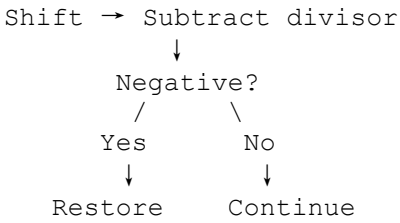
Thus, correct sign extension and arithmetic shifting are essential for obtaining accurate signed multiplication results.

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

In the restoring division algorithm, the partial remainder is shifted and the divisor is subtracted. If the result becomes negative, the divisor is added back to restore the previous value.

This restoration operation is the main source of inefficiency.

The basic process is:



When the subtraction produces a negative result, the divisor must be added again. This creates an additional arithmetic operation and increases execution time.

Debugging the Problem

The unnecessary operation is the **restoration step**. Instead of immediately restoring the previous remainder, the algorithm can keep the negative partial remainder and use it in the next step.

Optimized Solution – Non-Restoring Division

In non-restoring division, restoration is avoided.

The algorithm works as follows:

- If the current remainder is positive, subtract the divisor.
- If the current remainder is negative, add the divisor.
- Perform the required shift after each step.
- Determine the quotient bit based on the sign of the partial remainder.

At the end, if the final remainder is negative, the divisor is added once to obtain the correct remainder.

Advantages

Non-restoring division reduces the number of add/subtract operations because it does not restore the remainder after every negative result.

Therefore:

Restoring division → more operations → higher execution time

Non-restoring division → fewer operations → better performance

Testing

The method should be tested with:

- Exact division
- Division with remainder
- Dividend smaller than divisor
- Different sign combinations

Thus, non-restoring division provides a more efficient approach for processor-based arithmetic operations.

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

IEEE 754 single precision uses **32 bits**:

- 1 bit for sign

- 8 bits for exponent
- 23 bits for fraction

When a very small number is added to a very large number, the main problem occurs during **exponent alignment**.

Before adding two floating-point numbers, their exponents must be made equal. The mantissa of the number with the smaller exponent is shifted to the right.

For example, if one number has a much larger exponent, the smaller number may have to be shifted by many positions. After enough shifting, its significant bits may be lost.

Therefore, although the small number is mathematically present, it may have little or no effect on the final result.

Debugging the Error

The main causes are:

1. Difference in exponents.
2. Loss of significant bits during alignment.
3. Rounding after addition.
4. Limited precision of single-precision representation.
5. Normalization of the final result.

After addition, the result must be normalized and rounded according to IEEE 754 rules. Rounding can introduce a small error.

Solution

One simple solution is to use **double precision** instead of single precision. Double precision provides more bits for the fraction and exponent, so it can represent values with greater accuracy.

Another solution is to improve the algorithm by reducing repeated floating-point operations and using suitable numerical techniques.

For example, instead of repeatedly adding very small values to a large value, the small values can be accumulated separately and added later. This can reduce loss of precision.

Testing

The program should be tested using:

- Two numbers of similar magnitude
- Very large + very small number
- Very small + very small number
- Positive and negative values

The results should also be compared between single and double precision.

Thus, exponent alignment, normalization and rounding are important causes of floating-point inaccuracies. Using double precision and better numerical algorithms can improve the accuracy of the scientific application.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	**Debugging Approach & Analysis (20%)**	**Correctnes s of Debugged Solution (25%)**	**Optimization Techniques Applied (20%)**	**Testing and Documentation (15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem(4)	Identifies most errors with minor omissions(3)	Identifies limited errors; misses key issues(2)	Unable to identify errors or misunderstands the problem(1)
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification(4)	Applies suitable debugging methods with minor gaps(3)	Uses basic debugging methods with limited analysis(2)	No systematic debugging approach followed(0)
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output(5)	Provides working solution with minor errors(4)	Partially correct solution with limited functionality(3)	Incorrect solution or fails to resolve errors(0)
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality(4)	Performs optimization with noticeable improvements(3)	Limited optimization with minimal improvements(2)	No optimization applied or inefficient implementation(0)
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation(3)	Provides adequate testing and documentation	Limited testing and incomplete documentation(1)	No proper testing or documentation provided(0(

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
			with minor issues(2)		

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	**CSA1223**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL3-. DEBUGGING & OPTIMISATION TASK**
Weightage:	**20%**	**Total Marks:**	**100**
CO2 Statement:	*Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)*		
Student Name:	R.DINESH KUMAR	**Reg.No.:**	192312258
Department:	B.E ECE	**Date:**	12/08/26

89

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I R. Dinesh Kumar [192312258] *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Dinesh Kumar

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)	
Q1	4	3	5	3	2	17
Q2	3	4	4	4	2	17
Q3	4	3	3	2	3	15
Q4	3	4	4	4	2	17
Q5	4	3	4	3	1	15

81

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem(4)	Identifies most errors with minor omissions(3)	Identifies limited errors; misses key issues(2)	Unable to identify errors or misunderstands the problem(1)
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification(4)	Applies suitable debugging methods with minor gaps(3)	Uses basic debugging methods with limited analysis(2)	No systematic debugging approach followed(0)
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output(5)	Provides working solution with minor errors(4)	Partially correct solution with limited functionality(3)	Incorrect solution or fails to resolve errors(0)
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality(4)	Performs optimization with noticeable improvements(3)	Limited optimization with minimal improvements(2)	No optimization applied or inefficient implementation(0)
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation(3)	Provides adequate testing and documentation with minor issues(2)	Limited testing and incomplete documentation(1)	No proper testing or documentation provided(0(